

3교시: 인프라 엔지니어가 MSA에서 알아야 할 것

Week 3 Day 1 Lesson 3

인프라 엔지니어가 MSA에서 알아야 할 운영 계약

개발팀과 인프라팀의 약속을 문서화하여 안정적인 배포와 운영을 보장합니다.



운영 계약

서비스가 운영 환경에서 안정적으로 동작하기 위해 개발팀이 제공해야 하는 정보와 약속입니다.

서비스 목록 (서비스명)	이미지 (컨테이너 이미지)	포트 (서비스 포트)	프로토콜 (통신 프로토콜)	환경변수 (필수 환경변수)	의존성 (외부 의존 서비스)	health (헬스 체크 엔드포인트)	logs (로그 확인 명령어)	개발팀 전달사항 (운영 담당자/연락처)
order-service	order-service:1.0.0	8080	HTTP	DB_HOST DB_USER DB_PASSWORD LOG_LEVEL	mysql (redis)	GET /actuator/health	kubectl logs -f -app=order-service	이주연 (Order팀) 010-1111-2222
product-service	product-service:1.0.0	8080	HTTP	DB_HOST DB_USER DB_PASSWORD LOG_LEVEL	mysql (warehouse-service)	GET /actuator/health	kubectl logs -f -app=product-service	박민수 (Product팀) 010-2222-3333
user-service	user-service:1.0.0	8080	HTTP	DB_HOST DB_USER DB_PASSWORD JWT_SECRET LOG_LEVEL	mysql (redis)	GET /actuator/health	kubectl logs -f -app=user-service	최수빈 (User팀) 010-3333-4444
payment-service	payment-service:1.0.0	9090	HTTP	PG_URL PG_KEY LOG_LEVEL	external-pay-gateway (redis)	GET /actuator/health	kubectl logs -f -app=payment-service	정현우 (Payment팀) 010-4444-5555
delivery-service	delivery-service:1.0.0	8081	HTTP	MAPS_API_KEY LOG_LEVEL	maps-api (notification-service)	GET /actuator/health	kubectl logs -f -app=delivery-service	김도윤 (Delivery팀) 010-5555-6666

운영 계약 체크리스트

- ☒ 서비스명과 컨테이너 이미지 태그가 정확히 정의되었는가?
- ☒ 포트와 프로토콜(HTTP/HTTPS/gRPC 등)이 명시되었는가?
- ☒ 필수 환경변수가 빠짐없이 정의되었는가?
- ☒ 외부 의존 서비스와 의존 방향이 정확한가?
- ☒ 헬스 체크 엔드포인트가 구현되어 있고 응답 기준이 합의되었는가?
- ☒ 로그 확인 명령어가 제공되었는가?
- ☒ 운영 담당자와 비상 연락처가 명시되었는가?

개발팀 전달사항 (운영에 필수!)

- 배포 문서**
빌드/배포 절차, 마이그레이션(필요시), 롤백 절차
- 설정 정보**
필수 환경변수 목록, 기본값, 예시
- 장애 대응 가이드**
자주 발생하는 장애와 복구 방법
- 모니터링 지표**
핵심 비즈니스 지표, 애플리케이션 지표

왜 운영 계약이 중요한가?

- 배포 속도 향상:** 필요한 정보가 명확하여 반복 질문 감소
- 장애 대응 시간 단축:** 로그 위치, 헬스 체크로 빠른 원인 파악
- 책임 소재 명확:** 연락처와 소유권으로 의사결정 단순화
- 표준화된 운영:** 모든 서비스가 동일한 기준으로 관리
- 지식 공유 용이:** 문서화된 정보로 팀 변경 시에도 안정적 운영

TIP

운영 계약은 코드 변경 시 함께 업데이트하세요. (새로운 환경변수 추가, 포트 변경, 의존성 변경 등)

개발 완료

→

운영 계약 업데이트

→

인프라 검토

→

배포

→

운영

→

수업 목표

- MSA에서 인프라/DevOps 엔지니어가 알아야 할 service contract를 정리한다.
- service별 image/build, port, env, dependency, health, log 위치를 표로 작성한다.
- 개발 코드 내부가 아니라 운영 증거를 기준으로 정상/비정상을 구분한다.

핵심 질문

MSA 운영자는 모든 비즈니스 로직을 다 알 필요는 없다. 하지만 다음은 반드시 알아야 한다.

- 이 service는 어떤 image로 실행되는가?
- 어떤 port를 열고 누구에게 공개하는가?
- 어떤 environment variable이 없으면 실패하는가?
- 어떤 service에 의존하는가?
- 정상 상태는 어떤 endpoint나 log로 확인하는가?
- data를 어디에 저장하는가?

Service Contract 표

Day1 실습 앱을 기준으로 service contract를 작성한다.

Service	실행 기준	Port	Env	Dependency	Health/Log
frontend	nginx:1.27-alpine	host 18083 -> container 80	없음	api	nginx access/error log
api	build: ./api	expose 8080, host 18084	DB_HOST, DB_PORT	db	/health, api JSON log
worker	build: ./worker	host 공개 없음	API_URL, WORKER_INTERVAL_SECONDS	api	worker JSON log
db	postgres:16-alpine	host 공개 없음, internal 5432	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	volume	pg_isready, db log

이 표는 문서 장식이 아니다. 장애가 나면 바로 확인 순서가 된다.

compose.yaml에서 contract 찾기

```
api:
  build: ./api
  environment:
    SERVICE_NAME: api
    DB_HOST: ${DB_HOST:-db}
    DB_PORT: ${DB_PORT:-5432}
  expose:
    - "8080"
  ports:
    - "18084:8080"
  depends_on:
    db:
      condition: service_healthy
  healthcheck:
    test: ["CMD", "python", "-c", "... /health ..."]
```

읽는 법:

줄	의미
build: ./api	API image는 local Dockerfile로 만든다.
DB_HOST: db	API container는 DB를 service name db로 찾는다.
expose: 8080	Compose network 내부에서 8080을 사용한다.
ports: 18084:8080	강의 debug용으로 host에서도 API를 직접 확인한다.
condition: service_healthy	DB healthcheck가 통과된 뒤 API를 시작하려는 의도다.
healthcheck	container running이 아니라 API readiness를 확인한다.

운영자가 모르면 생기는 문제

모르는 것	생기는 문제
host port와 container port	curl 대상이 틀려서 정상 service를 장애로 오판
service name DNS	container 내부에서 localhost로 DB를 찾으려다 실패
env default	.env 값이 바뀌었는데 compose config를 안 보고 넘어감
healthcheck 의미	running인데 준비 안 된 service를 정상으로 오판
volume lifecycle	down -v로 DB data를 날림

실습

```
cd week3/day1/labs/msa-demo
docker compose config > /tmp/w3d1-compose-config.txt
```

다음 값을 찾아 표에 적는다.

찾을 값	내 답
frontend published port	
api container port	
api DB host	
worker API URL	
db volume name	
db healthcheck command	

Evidence Note

```
# W3D1S3 Service Contract
| service | image/build | port | env | dependency | health/log |
|---|---|---|---|---|---|
| frontend | | | | | |
| api | | | | | |
| worker | | | | | |
| db | | | | | |
```

핵심 포인트

MSA에서 인프라 엔지니어가 먼저 작성해야 하는 문서는 멋진 아키텍처 소개가 아니라 service contract 표다. 이 표가 있어야 장애 상황에서 어디를 먼저 볼지 결정할 수 있다.